

# Reflective Programming

## An Annotated Bibliography

Jania Vandevoorde (janiavdv@umich.edu)  
University of Michigan

April 20, 2026

### Narrative

When considering data in programs, programmers often focus on external inputs like files, user input, and network packets. Reflection in programming languages is the ability of a program to operate on itself, including inspecting and potentially modifying its own structure. This can happen statically or dynamically, each enabling many powerful features that would otherwise require invasive changes to a language’s implementation. For example, reflection can be used to build debuggers that analyze and execute code within the same language, or to do unbundling and reformatting of network data. This shifts the notion of “data” to include a program’s own structure and behavior, motivating a deeper study of reflective systems.

Explorations in the field begin with Brian Cantwell Smith’s 1984 paper introducing **procedural reflection** in 3-Lisp, a dialect of Lisp. His main insight was that a Lisp interpreter written in Lisp, or a metacircular interpreter, could be exposed to the program as a data structure to create what he called the **reflective tower**. Each level in his reflective tower consisted of a pair of the program and a representation of the interpreter, and each level processes the program below it. In 3-Lisp, the programmer calls a reflective procedure to climb the tower and inspect the interpreter itself. Smith’s paper opens the question of “how much should a program be able to know about itself?” and suggests that 3-Lisp has given the language too much access to its own structure, especially in the case of first-class functions.

Two seminal papers, both presented at OOPSLA 1987, extended Smith’s ideas into the object-oriented setting. First, Pattie Maes introduced the **metaobject protocol**, where every base object is paired with a metaobject that holds all the reflective information about it, such as its name,

instantiation method, message handler, and member variables. Metaobjects reify the interpreter, so modifying them during execution changes what the interpreter does. Then, Pierre Cointe addressed a weakness left open by Smith and Maes: metaobjects had remained anonymous and inaccessible to programmers. Cointe's ObjVlisp gave metaclasses first-class status by fusing classes, metaclasses, and object instances into a single object model that only differentiated between them by their inheritance patterns. Together, these papers establish the theoretical vocabulary on which the rest of the work in this field builds.

One of the challenges in the field until this point was bringing these ideas out of Lisp dialects into mainstream compiled languages like C++. Shigeru Chiba's 1995 work on Open C++ addressed this by moving reflection to compile time. Rather than having metaobjects at runtime like in the previous Lisp methods, Open C++ metaobjects walk the parse tree during a step before compilation called translation where substitutions turn the code into regular C++. One advantage of this approach was that it required absolutely no changes in the C++ compiler or runtime system. It was a major breakthrough, however, it made runtime structural reflection in compiled languages impossible.

Five years later, Chiba introduced Javassist. This new framework for Java added **structural reflection**, or the ability to not only see program elements using reflection but modify them too. Rather than being a compile time process like Open C++ and Open Java, Javassist intercepts classes as they're loaded to the Java Virtual Machine (JVM), allowing for a more dynamic version of reflection. In the paper's code example benchmarks, Javassist is over 10 times faster than existing approaches. Although this was a powerful addition to making reflection more dynamic in compiled languages, Javassist still lacks the ability to perform reflective operations on a running JVM.

Bracha and Ungar's 2004 paper took a step back from implementation to ask a more philosophical design question: why do reflective API's designs keep tangling meta operations into base objects? They defined three major principles for how reflective systems should be implemented, which fall into the same school of thought as Maes had back in 1987. Firstly, *encapsulation* entails that reflective functionality should not be accessible through base objects themselves. Next, *stratification* means that the meta level should be cleanly separated from the base level. Finally, *ontological correspondence* states that the constructs at the meta level should mirror the structure of what they describe at the base level. Based on these principles, the authors argued that they are hard to satisfy simultaneously (in fact, this is one of

the shortcomings of the paper), but **mirrors** are the best way to do so. Mirrors are dedicated reflective objects created through a separate factory rather than through the base object itself. The approach was well evaluated and this paper is often cited as a foundation for what a reflective system should be. Though its ideas are largely tied to Maes' original proposal, the principles hadn't been defined this explicitly, so this paper laid the ground work for modern reflective systems.

Even with all these advancements in reflective power that modern languages were capable of, performance remained a key obstacle in this field. All prior approaches had to sacrifice either expressive power to be efficient or efficiency to be expressive. This includes having to label each part of the program that is reflective, a huge burden on the programmer.

In 2015, Marr, Seaton, and Ducasse resolved this with a new data structure called **dispatch chains**, implemented entirely at the program's runtime level so that the programmer had no changes to make. Reflective lookups such as method calls or field accesses are cached on their first use and then reused from the cache after. The paper's benchmark results showed that reflective programs could compile to machine code identical to their equivalent non-reflective version. They also observed a 10-20x speedup in image processing libraries in Smalltalk and JRuby, compared to prior reflection implementations. Of course, this could always be improved upon and this paper only adds these runtime checks for these specific languages. It also doesn't address the memory overhead than any caching approach introduces.

In conclusion, these seven papers follow a coherent 30 year arc, from reflection's theoretical foundations in Lisp, through object-oriented programs, to practical deployment in mainstream languages, and finally performance enhancements. There remain several open questions in the field. First and foremost, there is no correct answer to how much structural access a program should have to itself. The papers discussed are merely attempts to answer that question, but it will remain open as long as programming languages exist. Secondly, Marr et al. explicitly acknowledge that their dispatch chained approach has only been validated for self-optimizing JIT compiled interpreters, so it remains to be discovered whether this can be generalized to all languages. Finally, while the mirror principles from Bracha and Ungar's paper are widely cited, no existing language fully satisfies all three principles simultaneously.

## Implementation

I am implementing a reflective recursive ALFA dialect in OCaml. So far, it is very inspired by the original Smith paper with 3-Lisp. A target goal is being able to write a toy example where a program inspects whether its own argument is an addition before evaluating it.

## References

- [1] G. Bracha and D. M. Ungar, “Mirrors: design principles for meta-level facilities of object-oriented programming languages,” in *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2004, October 24-28, 2004, Vancouver, BC, Canada*, J. M. Vlissides and D. C. Schmidt, Eds. ACM, 2004, pp. 331–344. [Online]. Available: <https://doi.org/10.1145/1028976.1029004>

Bracha and Ungar take a more philosophical approach to addressing a design problem that has persisted across all the prior work in the area: reflective APIs mix meta operations into base objects. For example, in Java, objects have a `getClass()` method, meaning the same class that serves method calls like `increment()` or `get()` can also answer questions about its own structure. There are several concerns associated with this, including security problems. The paper proposes three big design principles for reflection APIs: encapsulation (reflective functionality should not be accessible through base objects themselves), stratification (the meta level should be cleanly separated from the base level), and ontological correspondence (the constructs available at the meta level should mirror the structure of what they describe at the base level). Then, it argues that the principles are satisfied by building reflection around what the authors call “mirrors.” A mirror is a separate object with reflective functionality that is created through a dedicated factory, not through the base object itself. These ideas are strongly related to the ideas of Maes back in 1987. The paper evaluates by comparing traditional reflection against its mirror-based approach, finding that mirror-based APIs handle distribution and deployment significantly better than traditional reflection, since the clean separation between base

and meta levels means reflection can be swapped out or removed entirely without touching application code. The paper acknowledges that no existing system fully satisfies all three principles it has defined and leaves this as future work.

- [2] S. Chiba, “A metaobject protocol for C++,” in *Proceedings of the Tenth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA 1995, Austin, Texas, USA, October 15-19, 1995*, R. Wirfs-Brock, Ed. ACM, 1995, pp. 285–299. [Online]. Available: <https://doi.org/10.1145/217838.217868>

After Maes and Cointe established metaobjects and first-class metaclasses in Lisp dialects, the open problem was bringing reflective power to mainstream compiled languages. Some prior designs relied on metaobjects being active at runtime, so they were incompatible with the static compilation model of popular languages like C++. Furthermore, these runtime approaches added a ton of overhead, whereas Chiba recognized that by making reflection a compile-time process, this overhead could be significantly reduced. Chiba introduced a new architecture where metaobjects control the compilation of programs rather than changing runtime processes. This is implemented as an extension to C++ called Open C++, which requires no changes to the compiler or runtime system. The programmer writes Open C++ code which can include reflective functionality. Then, the Open C++ is translated to regular C++ before compile time. This translation happens using a metaobject for each class and member function, which does its own translation by walking the parse tree and substituting code fragments. The paper evaluates Open C++ by providing four examples of code that previously would have required additional programmer efforts, compiler optimizations, or runtime costs, but is now packaged into a new library that only slightly increases compile times. The paper’s limitation is that since the metaobjects exist only during compilation and are immediately discarded after, they cannot respond to runtime state, so we can’t achieve dynamic structural reflection. This motivates Chiba’s later work in 2000.

- [3] —, “Load-time structural reflection in java,” in *ECOOP 2000 - Object-Oriented Programming, 14th European Conference, Sophia Antipolis*

and Cannes, France, June 12-16, 2000, *Proceedings*, ser. Lecture Notes in Computer Science, E. Bertino, Ed., vol. 1850. Springer, 2000, pp. 313–336. [Online]. Available: [https://doi.org/10.1007/3-540-45102-1\\_16](https://doi.org/10.1007/3-540-45102-1_16)

By 2000, Java had introspective reflection implemented in its `java.lang.reflect` API. However, it lacked the ability for the programmer to modify the program behavior (behavioral reflection). Chiba’s paper specifically addresses this by introducing a system called Javassist that is implemented without having to modify the runtime system or the compiler. Javassist intercepts a class before it is loaded in the JVM, transforms its bytecode according to the structural changes, and then loads the modified class normally. This approach is notably different from existing behavioral reflection attempts in Java where those implementations required modifications to a running Java Virtual Machine. The paper evaluates by going through three examples: binary code adaptation, behavioral reflection, and remote method invocation. It benchmarks the implementation against OpenJava, a comparable system that works at the source code level rather than bytecode, showing that Javassist is over 10 times faster. The main limitation of Chiba’s work is that structural reflection is only allowed before a class is loaded into a JVM, it cannot be changed at runtime.

- [4] P. Cointe, “Metaclasses are first class: the objvlist model,” in *Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA 1987, Orlando, Florida, USA, October 4-8, 1987, Proceedings*, N. K. Meyrowitz, Ed. ACM, 1987, pp. 156–167. [Online]. Available: <https://doi.org/10.1145/38765.38822>

This paper appears with Maes’ at OOPSLA 1987. In it, Cointe addresses the problem left open by Smith and Maes. Instead of treating metaobjects as second class (where they’re anonymous and cannot be defined explicitly by users), Cointe’s ObjVlist introduces first class metaclasses. The solution fuses classes, metaclasses, and instances into a single kind of object. The main idea is that everything is just an object, and whether it’s a class or a metaclass is determined purely by inheritance. The system is defined with a special object called **Class**, which is defined as its own instance to avoid infinite inheritance checks. Then, for other objects, if they inherit the

new method, they can create instances and therefore are a class, otherwise they are an object (terminal instance). This means we can create as many metaclass levels as needed, when needed. The paper evaluates its idea through code examples, showing how ObjVlisp can simulate object-oriented languages like Smalltalk and Loops outside an object-oriented setting. It shows how standard object-oriented features (such as inheritance and method lookup) and reflective features come from the same kernel. It identifies future work as implementing the architecture within Smalltalk itself. Furthermore, the paper doesn't address performance, a clear limitation throughout all of the work in this field.

- [5] P. Maes, "Concepts and experiments in computational reflection," in *Conference on Object-Oriented Programming Systems, Languages, and Applications, OOPSLA 1987, Orlando, Florida, USA, October 4-8, 1987, Proceedings*, N. K. Meyrowitz, Ed. ACM, 1987, pp. 147–155. [Online]. Available: <https://doi.org/10.1145/38765.38821>

Maes extends Smith's procedural reflection into the object-oriented setting. The paper faces the problem of implementing metaobjects in an object-oriented language: a new layer of objects above base objects that describe the base objects. Maes theoretically introduces the metaobject protocol in procedural reflection where metaobjects contain all of the reflective information available about the base object. For example, a metaobject could contain the object's name, print method, message handling method, instantiation method, and so on. They are only constructed when they're actually needed. The biggest difference here from Smith's procedural reflection is that now the metaobjects are strictly separated from the base objects. During program interpretation, the metaobjects themselves would reify the interpreter, so changing them using reflection during the program would change what it interprets to, as intended. An implementation of this would help answer the question raised in Smith's paper about how much a program should be able to know about itself. However, the paper acknowledges that actually implementing structural reflection remains an open problem. The core challenge is that keeping a separate self-description in tandem with the object requires

a more intelligent interpreter. Maes’ experiments in this paper fall back on procedural reflection, sharing the limitations that Smith introduced.

- [6] S. Marr, C. Seaton, and S. Ducasse, “Zero-overhead metaprogramming: reflection and metaobject protocols fast and without compromises,” in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, D. Grove and S. M. Blackburn, Eds. ACM, 2015, pp. 545–554. [Online]. Available: <https://doi.org/10.1145/2737924.2737963>

Through all the prior papers in this field, performance remained an open problem. Historically, attempts to fix this had to give up either expressive power or required manual programmer effort. Marr’s solution was to cache results from reflective lookups the first time they’re performed so they could be reused in the future without having to do the lookup again. They use a “dispatch chain” data structure to do so. Crucially, they accomplished this optimization in the language implementation level, so there are no reductions to expressiveness or additional requirements of the programmer. The optimization was benchmarked in Smalltalk and then in JRuby with image processing code and it was shown that reflective programs compile to the same machine code as the non-reflective version. The optimized version yielded a 10-20x speedup with no changes to the image processing source code. However, the approach did introduce a small overhead due to the runtime checks required to verify cached data and the memory to represent the cache itself. The main limitation is that the results are only for self-optimizing interpreters that use JIT compilation. Whether dispatch chains generalize to other compiler architectures is left as future work. Furthermore, caching approaches always add a memory overhead, which the paper doesn’t address.

- [7] B. C. Smith, “Reflection and semantics in lisp,” in *Conference Record of the Eleventh Annual ACM Symposium on Principles of Programming Languages, Salt Lake City, Utah, USA, January 1984*, K. Kennedy, M. S. V. Deussen, and L. Landweber, Eds. ACM Press, 1984, pp. 23–35. [Online]. Available: <https://doi.org/10.1145/800017.800513>



This is the original paper that introduces the foundations of reflective systems, which Smith ends up calling “procedural reflection.” The architecture is proposed in 3-Lisp, a dialect of Lisp. The paper’s central problem is how to give a procedural language a representation of itself embedded within it that can be read *and* modified at runtime. Clumsy read-only versions of this existed before where the programmer had to specify which parts of the program would be inspected and when, passing around a representation of it. He first creates an inductive mapping from the syntactic to semantic domain. Then, Smith’s solution uses a metacircular interpreter (Lisp interpreter written in Lisp) to create what he calls the “reflective tower,” where each level is a program paired with an interpreter, and the interpreter is reified as the data on the next level up. The program now has self-referential access. He also shows that the tower can be finitely implemented because the metacircular interpreter itself and the programs its processes are tail-recursive. The paper’s evaluation includes proofs of semantic consistency and code examples to demonstrate the expressive power of the new 3-Lisp. It also discusses how the architectures are implementable in a variety of languages based on the work they did in Lisp. However, a limitation of the architecture noted in the paper is it perhaps gives too much metastructural access to computational constructs like functions and it leaves the question of how much a program should be able to know about its own structure open. Furthermore, no formal semantics or lambda calculus are provided for 3-Lisp. One additional limitation is that by adding this reflective tower, the complexity of program interpretation increases, but that may just be the tradeoff to gain reflective behavior.